

Hardware Engineering Lab

COMPLETE DRAFT DOCUMENTATION

FPGA-Based Traffic-Light Controller with Pedestrian Request Button

Team	A1
Team members	Md Riajul Islam Nadia Afruz Sayed Galib Hossen Rizvi Ronjon Sarker

Table of Contents

- Concept description
- Project/team management
- Implemented and integrated technologies
- System architecture and operating concept
- 6. VHDL and FPGA implementation
- 7. Testbench verification and simulation evidence
- 8. Synthesis, implementation, timing, utilization and power results
- 9. PCB design and hardware implementation
- 10. Discussion, problems faced and design decisions
- 11. Conclusion and future improvements
- Appendix A. VHDL source-file summary
- Appendix B. Nexys A7 pin mapping and XDC draft
- Appendix C. BOM summary
- Appendix D. Final review checklist

1. Concept description

The purpose of this Hardware Engineering Lab project is to design, verify and document an FPGA-based traffic-light controller for a simplified two-direction road intersection. The controller is described in VHDL, simulated with a testbench, synthesized and implemented in Vivado, and represented by a project-specific PCB design for the external traffic-light interface.

The system controls two vehicle directions, North-South (NS) and East-West (EW), using red, yellow and green LED outputs. The pedestrian crossing is represented by WAIT and WALK indicators. A pedestrian push button is the main human input. Because this button is asynchronous and mechanically noisy, it is synchronized, debounced and converted into a clean pulse before it is used by the controller.

The central design idea is a deterministic synchronous finite-state machine. Every state defines one safe output combination. The pedestrian request is stored in a `request_pending` flag and is served only after a safe all-red state is reached. Therefore, the pedestrian WALK signal is never active at the same time as any vehicle green or yellow movement.

FPGA-Based Traffic Light Controller with Pedestrian Request Top-Level Block Diagram

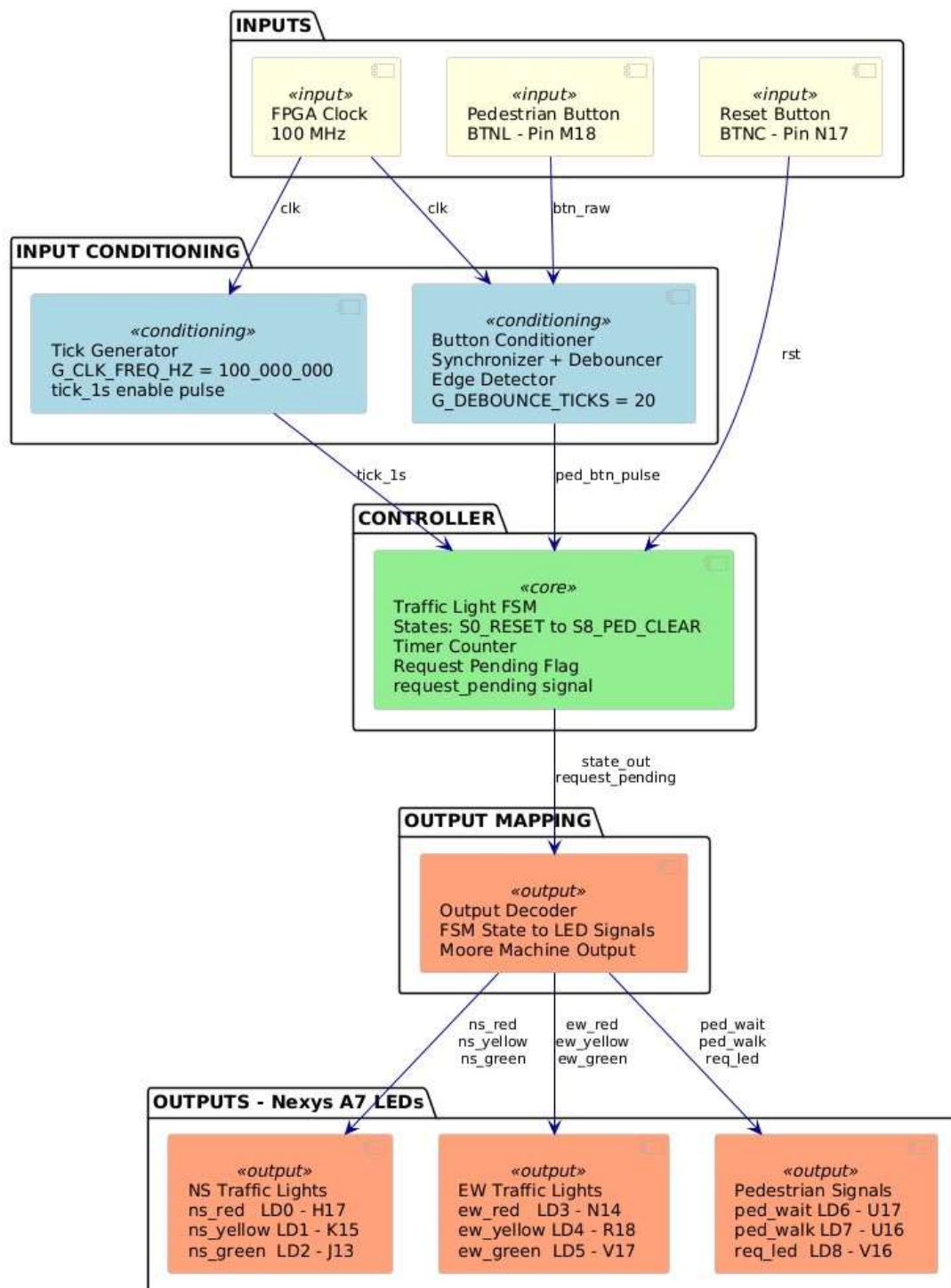


Figure 1: Top-level block diagram from the raw project sketch, showing inputs, input conditioning, controller core, output decoder and Nexys A7 LED assignments.

1.1 Project objectives

1. Design a clear state-based traffic-control algorithm for a two-direction intersection.
2. Implement the complete controller in synthesizable VHDL.
3. Synchronize and debounce the pedestrian-request push button.
4. Generate a slow timing tick from the 100 MHz FPGA board clock.
5. Verify reset behavior, normal sequence, pedestrian request behavior and safety constraints using a VHDL testbench.

6. Synthesize, implement and generate a bitstream for the Nexys A7 FPGA board in Vivado.
7. Create a PCB schematic, PCB layout, 3D view, BOM and Gerber outputs for the external traffic-light interface.
8. Document the design workflow, module structure, test evidence, implementation results and individual team contributions.

1.2 Functional requirements

ID	Requirement	Acceptance criterion
FR-01	The NS direction shall have RED, YELLOW and GREEN outputs.	Exactly one NS vehicle color is active in every normal vehicle phase.
FR-02	The EW direction shall have RED, YELLOW and GREEN outputs.	Exactly one EW vehicle color is active in every normal vehicle phase.
FR-03	NS GREEN and EW GREEN shall never be active simultaneously.	The testbench includes a safety assertion for this condition.
FR-04	The controller shall include one pedestrian request push button.	A debounced button press sets the stored request_pending flag.
FR-05	The request shall be served only in a safe all-red vehicle condition.	Pedestrian WALK becomes active only when NS and EW are RED.
FR-06	The controller shall include a reset input.	Reset returns the system to S0_RESET, an all-red startup state.
FR-07	The design shall use configurable timing constants.	Timing can be changed in the VHDL package without rewriting the FSM.
FR-08	The design shall be synthesizable and implementable on the Nexys A7.	Vivado synthesis, implementation and bitstream generation complete successfully.

2. Project/team management

The project is divided into digital-design tasks and PCB-design tasks. The digital flow starts with specifications and module partitioning, continues through VHDL coding and simulation, and ends with Vivado implementation and bitstream generation. The PCB flow starts with the schematic and component selection, continues through layout/routing, and ends with the BOM, 3D view and Gerber manufacturing files.

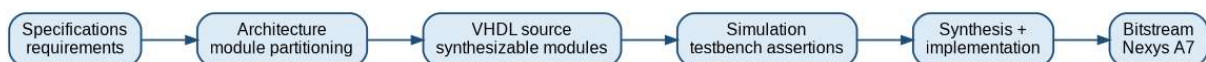


Figure 2: Digital-design workflow used for the VHDL and FPGA implementation.



Figure 3: PCB-design workflow used for the traffic-light interface board.

3. Implemented and integrated technologies

3.1 VHDL

VHDL is used to describe the traffic-light controller as synthesizable digital logic. The source files use IEEE.STD_LOGIC_1164 for standard logic signals and IEEE.NUMERIC_STD where numerical counters are needed.

3.2 FPGA and Nexys A7

The FPGA is the programmable hardware target for the VHDL logic. The Vivado project targets the Artix-7 part xc7a100tcsq324-1 and the Digilent Nexys A7-100T board. The board supplies the 100 MHz clock, push buttons and user LEDs required for the prototype.

3.3 Vivado 2025.2

Vivado is used for synthesis, implementation, timing analysis, utilization reporting, power reporting, waveform simulation and bitstream generation. The supplied project file and logs show Vivado v2025.2.

3.4 Altium Designer

The PCB design package is supplied as Altium schematic and PCB files. The PCB output package contains generated Gerber layers, a board PDF, a 3D view and a BOM spreadsheet.

3.5 GitHub / version control

A version-control workflow is recommended for the VHDL files, XDC constraints, PCB design files, documentation and report outputs so that design changes remain traceable.

4. System architecture and operating concept

The top-level VHDL design is structural. The traffic_light_top entity connects the tick generator, the button conditioner, the traffic-light FSM and the output decoder. The internal tick_1s signal controls phase durations. The internal s_btn_pulse signal represents a debounced one-clock request pulse. The FSM outputs the current state and the request_pending flag, and the output decoder maps these signals to board LEDs.

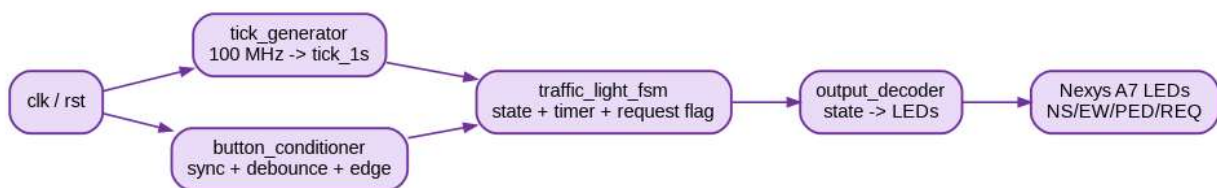


Figure 4: Implemented VHDL module structure based on the uploaded source files.

4.1 Traffic-light state machine

The traffic-light controller package defines nine states. S0_RESET is the startup all-red state. S1 to S6 form the normal NS/EW vehicle sequence with all-red clearance states. S7_PED_WALK serves the pedestrian request with all vehicle lights red. S8_PED_CLEAR returns the crossing to WAIT before the vehicle cycle resumes.

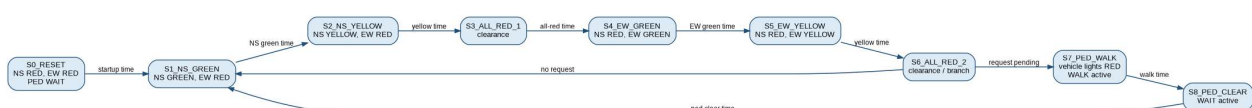


Figure 5: Actual FSM sequence implemented by traffic_light_fsm.vhd.

State	Vehicle outputs	Pedestrian outputs	Transition condition
S0_RESET	NS RED, EW RED	WAIT active	Startup timer expires
S1_NS_GREEN	NS GREEN, EW RED	WAIT active	NS green timer expires
S2_NS_YELLOW	NS YELLOW, EW RED	WAIT active	Yellow timer expires
S3_ALL_RED_1	NS RED, EW RED	WAIT active	All-red clearance timer expires
S4_EW_GREEN	NS RED, EW GREEN	WAIT active	EW green timer expires
S5_EW_YELLOW	NS RED, EW YELLOW	WAIT active	Yellow timer expires
S6_ALL_RED_2	NS RED, EW RED	WAIT active	If request_pending = 1, go to S7; otherwise return to S1
S7_PED_WALK	NS RED, EW RED	WALK active	Walk timer expires and request flag is cleared
S8_PED_CLEAR	NS RED, EW RED	WAIT active	Pedestrian-clearance timer expires

4.2 Timing constants

Constant	Value	Description
C_NS_GREEN_TIME	5	seconds/ticks in the current controller package
C_EW_GREEN_TIME	5	seconds/ticks in the current controller package
C_YELLOW_TIME	2	seconds/ticks in the current controller package
C_ALLRED_TIME	1	seconds/ticks in the current controller package
C_WALK_TIME	4	seconds/ticks in the current controller package
C_PEDCLEAR_TIME	2	seconds/ticks in the current controller package
C_STARTUP_TIME	1	seconds/ticks in the current controller package

5. VHDL and FPGA implementation

The design was implemented as separate VHDL modules to keep the controller easy to understand and test. Each module has one main responsibility, and the top-level entity connects the modules using internal signals.

File	Entity / package	Main responsibility
traffic_light_pkg.vhd	traffic_light_pkg	Defines the t_state enumeration and phase timing constants.

tick_generator.vhd	tick_generator	Counts 100 MHz clock cycles and produces a one-cycle tick_1s enable pulse.
button_conditioner.vhd	button_conditioner	Synchronizes the raw pedestrian button, debounces it and generates btn_pulse.
traffic_light_fsm.vhd	traffic_light_fsm	Stores the current state, timer count and request_pending flag; performs transitions.
traffic_light_output_decoder.vhd	traffic_light_output_decoder	Maps FSM states to NS, EW, pedestrian and request-status LED outputs.
traffic_light_top.vhd	traffic_light_top	Top-level structural integration for FPGA implementation.
traffic_light_fsm_tb.vhd	traffic_light_fsm_tb	Behavioral simulation testbench with reset, button stimulus and safety assertions.

5.1 Tick generator

The tick_generator module creates a slow enable signal from the board clock. It keeps the rest of the controller synchronous while allowing traffic phases to change at human-visible times.

```
entity tick_generator is
  generic (G_CLK_FREQ_HZ : integer := 100_000_000);
  port (clk : in std_logic; rst : in std_logic; tick_1s : out std_logic);
end entity tick_generator;
```

5.2 Button conditioner

The button_conditioner module protects the FSM from asynchronous and bouncing button transitions. It uses two synchronization flip-flops, a debounce counter and rising-edge detection to generate one clean request pulse.

```
entity button_conditioner is
  generic (G_DEBOUNCE_TICKS : integer := 20);
  port (clk : in std_logic; rst : in std_logic;
        btn_raw : in std_logic; btn_pulse : out std_logic);
end entity button_conditioner;
```

5.3 FSM request policy

The traffic_light_fsm module latches a request when ped_btn_pulse is asserted. The flag is not immediately allowed to interrupt an active vehicle green or yellow state. Instead, the request is served only from S6_ALL_RED_2, a safe all-red branch point. The flag is cleared inside S7_PED_WALK.


```

if ped_btn_pulse = '1' then
  r_req_pending <= '1';
end if;
...
when S6_ALL_RED_2 =>
  if r_timer >= C_ALLRED_TIME-1 then
    if r_req_pending = '1' then
      r_state <= S7_PED_WALK;
    else
      r_state <= S1_NS_GREEN;
    end if;
  end if;
when S7_PED_WALK =>
  r_req_pending <= '0';

```

5.4 Output decoder

The output decoder first sets a safe default of NS RED, EW RED and pedestrian WAIT. It then overrides only the relevant outputs for the active state. This approach makes all-red the default behavior for reset and unused states.

State group	Decoder behavior
Default / all-red states	NS RED = 1, EW RED = 1, PED WAIT = 1, all green/yellow outputs off.
S1_NS_GREEN	NS GREEN active; EW remains RED; pedestrian WAIT active.
S2_NS_YELLOW	NS YELLOW active; EW remains RED; pedestrian WAIT active.
S4_EW_GREEN	EW GREEN active; NS remains RED; pedestrian WAIT active.
S5_EW_YELLOW	EW YELLOW active; NS remains RED; pedestrian WAIT active.
S7_PED_WALK	Both vehicle directions remain RED; pedestrian WALK active; WAIT off.

5.5 Nexys A7 LED and button mapping

Signal	Function	Nexys A7 component / pin from raw sketch
clk	100 MHz FPGA clock	Board clock, 100 MHz
btn_ped	Pedestrian request button	BTNL - M18
rst	Reset button	BTNC - N17
ns_red	North-South red	LD0 - H17
ns_yellow	North-South yellow	LD1 - K15
ns_green	North-South green	LD2 - J13
ew_red	East-West red	LD3 - N14
ew_yellow	East-West yellow	LD4 - R18
ew_green	East-West green	LD5 - V17

ped_wait	Pedestrian WAIT	LD6 - U17
ped_walk	Pedestrian WALK	LD7 - U16
req_led	Stored request indicator	LD8 - V16

6. Testbench verification and simulation evidence

The uploaded testbench applies reset, allows the normal cycle to run, applies a pedestrian button press and then continues simulation so that the request can be observed in the waveform. The testbench also includes two safety assertions that fail the simulation if both vehicle greens are active together or if pedestrian WALK is active together with a vehicle green signal.

Test ID	Scenario	Expected result / evidence
TB-01	Reset is asserted for 100 ns.	The controller enters S0_RESET with NS RED, EW RED and pedestrian WAIT active.
TB-02	Normal traffic cycle runs.	NS and EW phases alternate with yellow and all-red intervals.
TB-03	Pedestrian button is pressed.	The request is captured by the conditioner/FSM and the req_led flag becomes active until the pedestrian phase begins.
TB-04	Pedestrian request is served.	Pedestrian WALK becomes active only after both vehicle directions are red.
TB-05	Continuous safety checking.	The testbench assertions check for simultaneous vehicle greens and pedestrian/vehicle conflicts.

6.1 Testbench safety assertions

```
assert NOT(tb_ns_grn='1' AND tb_ew_grn='1')
report "SAFETY FAIL: NS+EW green together!" severity FAILURE;

assert NOT(tb_ped_walk='1' AND (tb_ns_grn='1' OR tb_ew_grn='1'))
report "SAFETY FAIL: PED_WALK with vehicle green!" severity FAILURE;
```

6.2 Vivado simulation screenshots

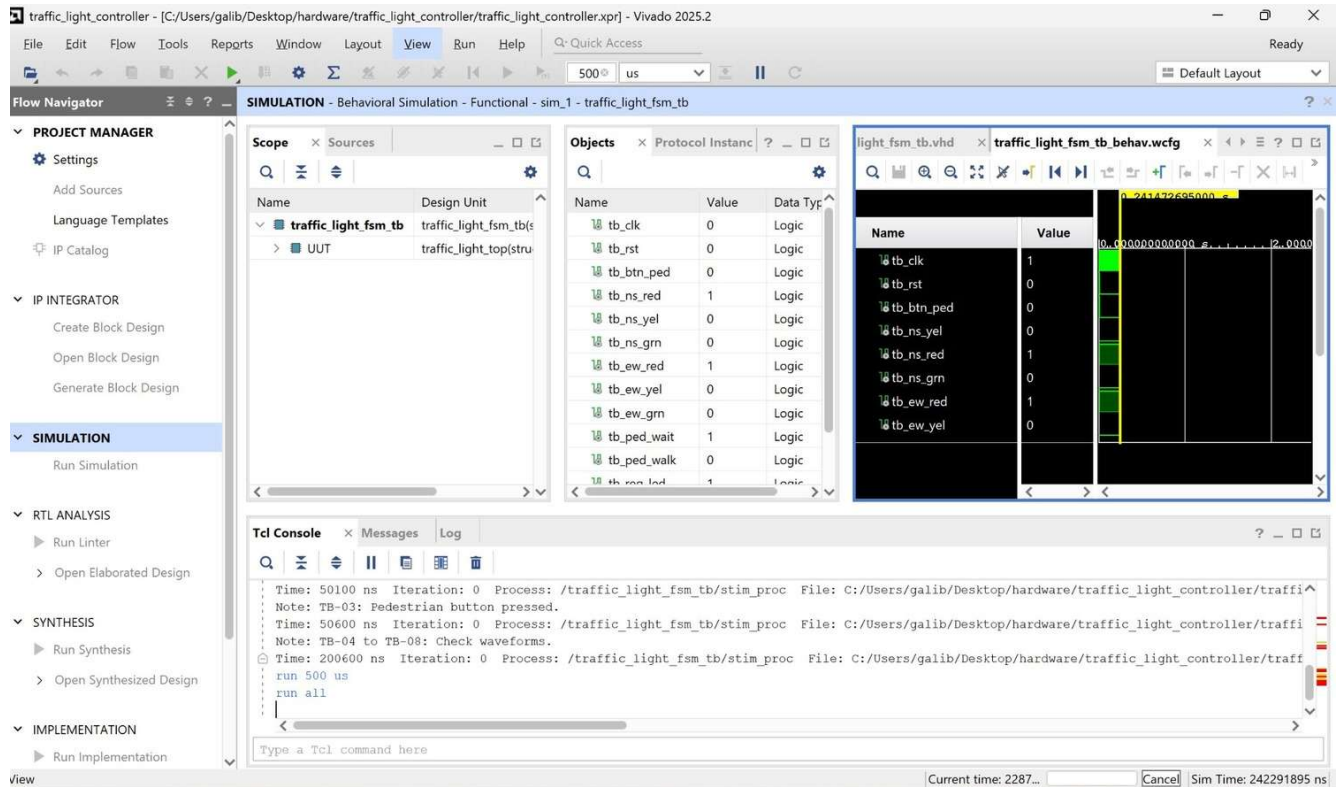


Figure 6: Behavioral simulation screenshot showing initial reset/all-red style output values and the active waveform window.

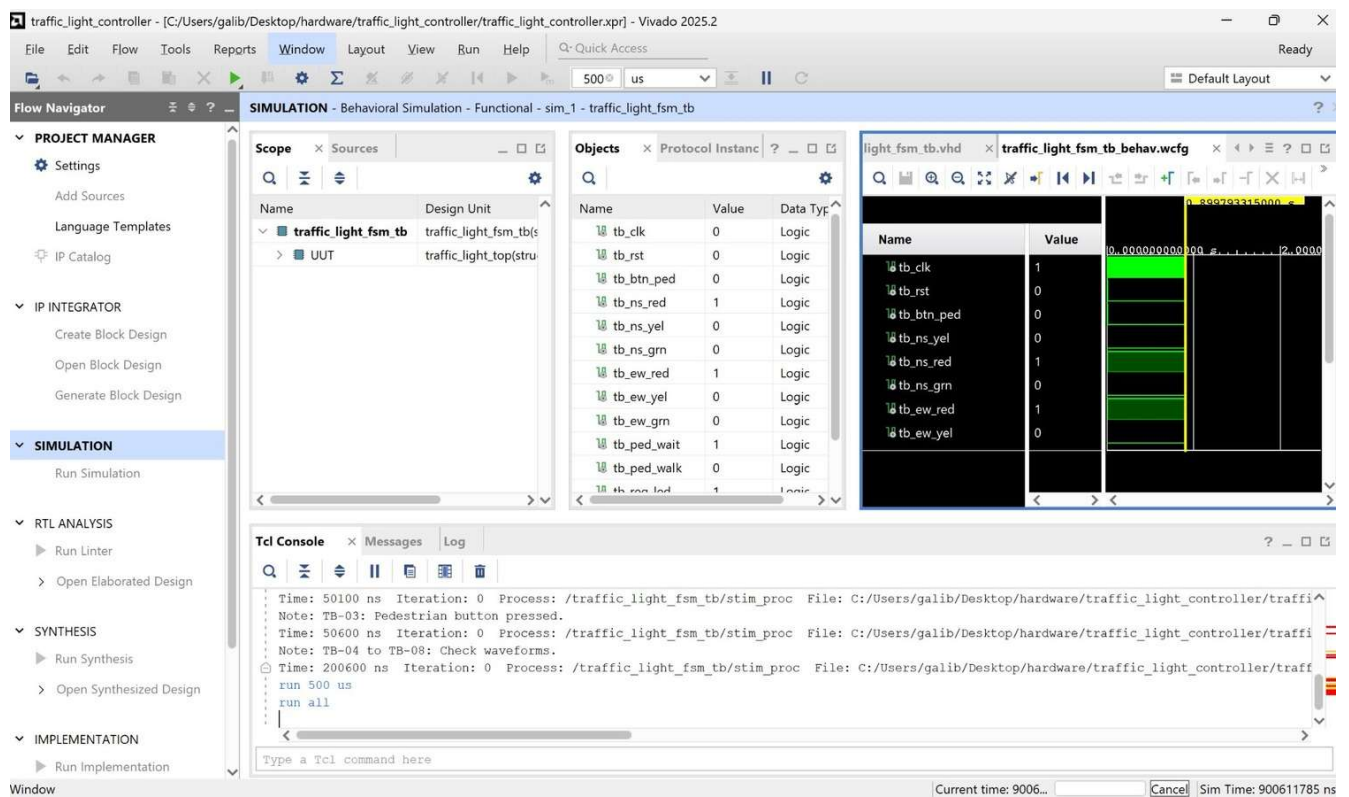


Figure 7: Behavioral simulation screenshot showing progression of output signals during the normal traffic-light sequence.

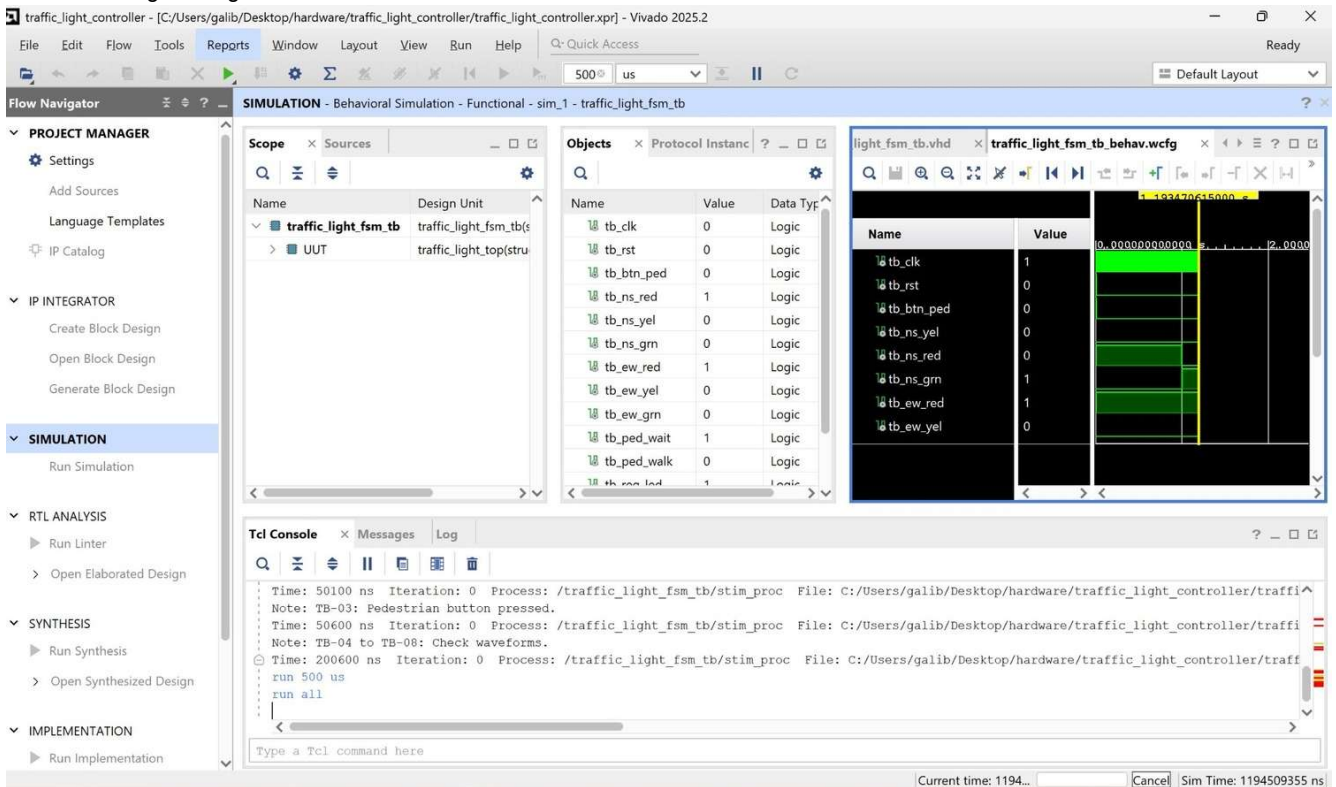


Figure 8: Behavioral simulation screenshot after the testbench reports a pedestrian button press and continues waveform checking.

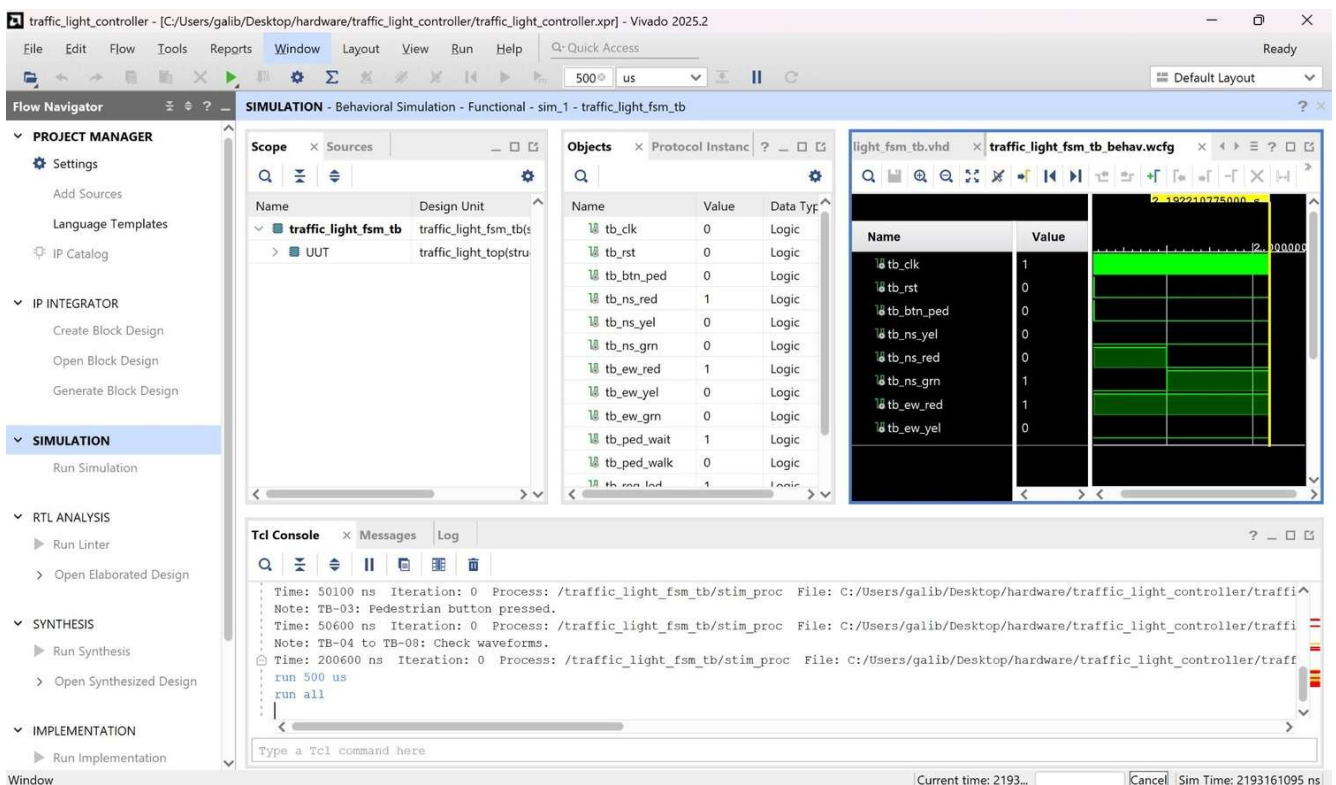


Figure 9: Later simulation waveform screenshot used as visual evidence that the state/output sequence continues after the request stimulus.

7. Synthesis, implementation, timing, utilization and power results

The Vivado project was synthesized and implemented for the Artix-7 target used on the Nexys A7-100T board. The reports show that the design is very small compared with the available resources and that the routed design meets the specified 100 MHz clock timing requirement.

Item	Result
Vivado version	Vivado v2025.2, 64-bit
Top-level design	traffic_light_top
Project target part	xc7a100tcs9324-1
Board part	digilentinc.com:nexys-a7-100t:part0:1.2
Design state for utilization/power/timing reports	Routed
Clock constraint	10.000 ns period, 100.000 MHz, waveform {0.000 5.000}
Bitstream status	Vivado GUI shows write_bitstream Complete

7.1 Timing result

Metric	Value	Interpretation
Worst Negative Slack (WNS)	4.144 ns	Positive setup slack; no setup timing failure.
Total Negative Slack (TNS)	0.000 ns	No accumulated setup violation.
Setup failing endpoints	0	No failing setup endpoints.
Worst Hold Slack (WHS)	0.132 ns	Positive hold slack; no hold timing failure.
Total Hold Slack (THS)	0.000 ns	No accumulated hold violation.
Hold failing endpoints	0	No failing hold endpoints.
Worst Pulse Width Slack (WPWS)	4.500 ns	Positive pulse-width slack.
Pulse-width failing endpoints	0	No failing pulse-width endpoints.

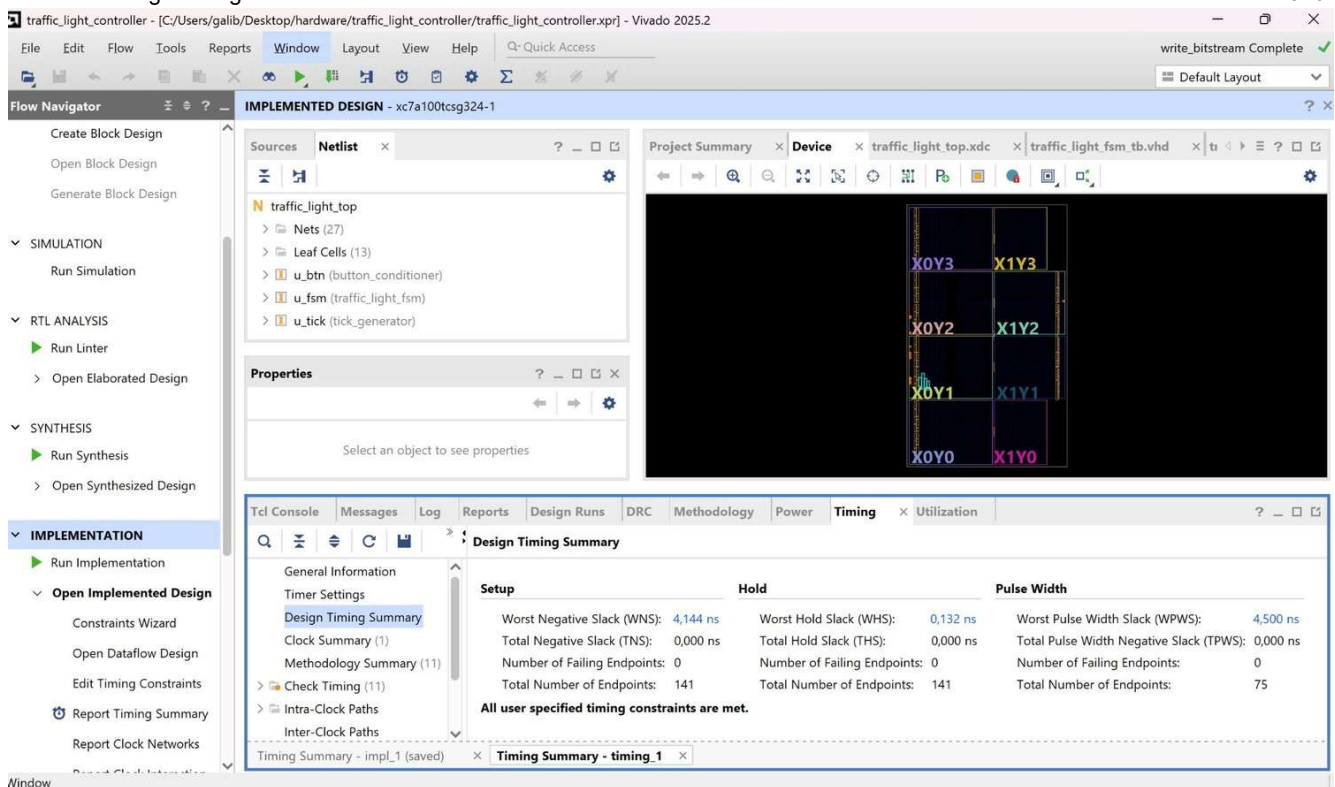


Figure 10: Vivado implemented-design timing summary showing positive WNS, WHS and WPWS with zero failing endpoints.

7.2 Utilization result

Resource	Used	Available	Utilization
Slice LUTs	110	63400	0.17
Slice Registers	74	126800	0.06
Slices	110	63400	0.17
Bonded IOB	12	210	5.71
BUFGCTRL	1	32	3.13
DSP	0	240	0.00
Block RAM Tile	0	135	0.00

Implementation insight	Explanation
Low utilization	The controller uses only 110 LUTs and 74 flip-flops in the routed report. This confirms that the design is small and suitable for the Nexys A7-100T.
No BRAM/DSP usage	The design is pure control logic, so it does not require block RAM or DSP resources.
IO usage	The 12 bonded I/O ports correspond to clock/reset/button inputs and LED outputs.

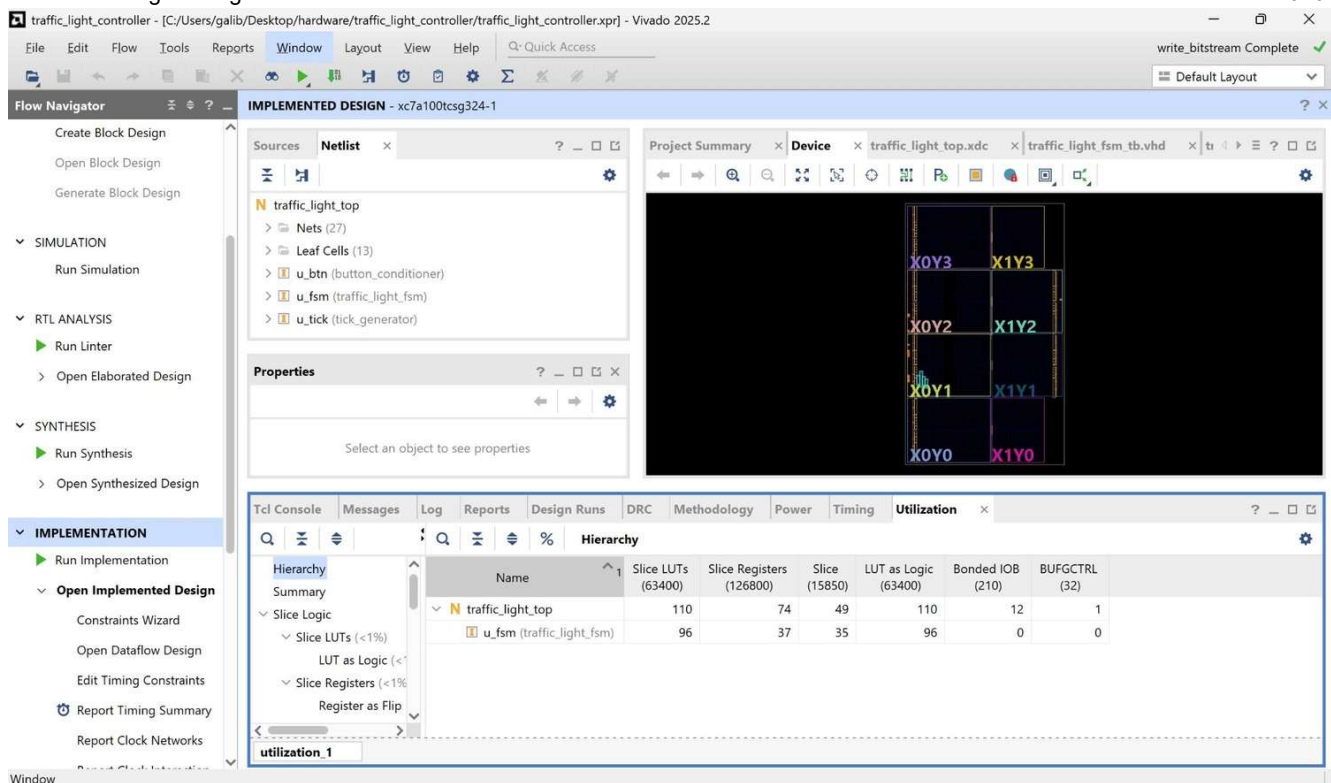


Figure 11: Vivado implemented-design utilization screenshot showing low LUT, register and I/O usage.

7.3 Power result

Power metric	Value	Comment
Total on-chip power	0.098	Estimated routed-design power.
Dynamic power	0.001	Low because the design is mostly simple control logic.
Device static power	0.097	Dominant component of total power.
Junction temperature	25.4	Estimated junction temperature at the report settings.
Maximum ambient	84.6	Estimated maximum ambient from the report.
Confidence level	Medium	Activity estimation is not fully simulation-annotated.

7.4 Bitstream evidence

The Vivado GUI screenshot shows `write_bitstream Complete`. This proves that the implementation flow reached bitstream generation. The hardware manager panel in the screenshot is unconnected, so the screenshot should be treated as bitstream-generation evidence, not as separate proof of board programming.

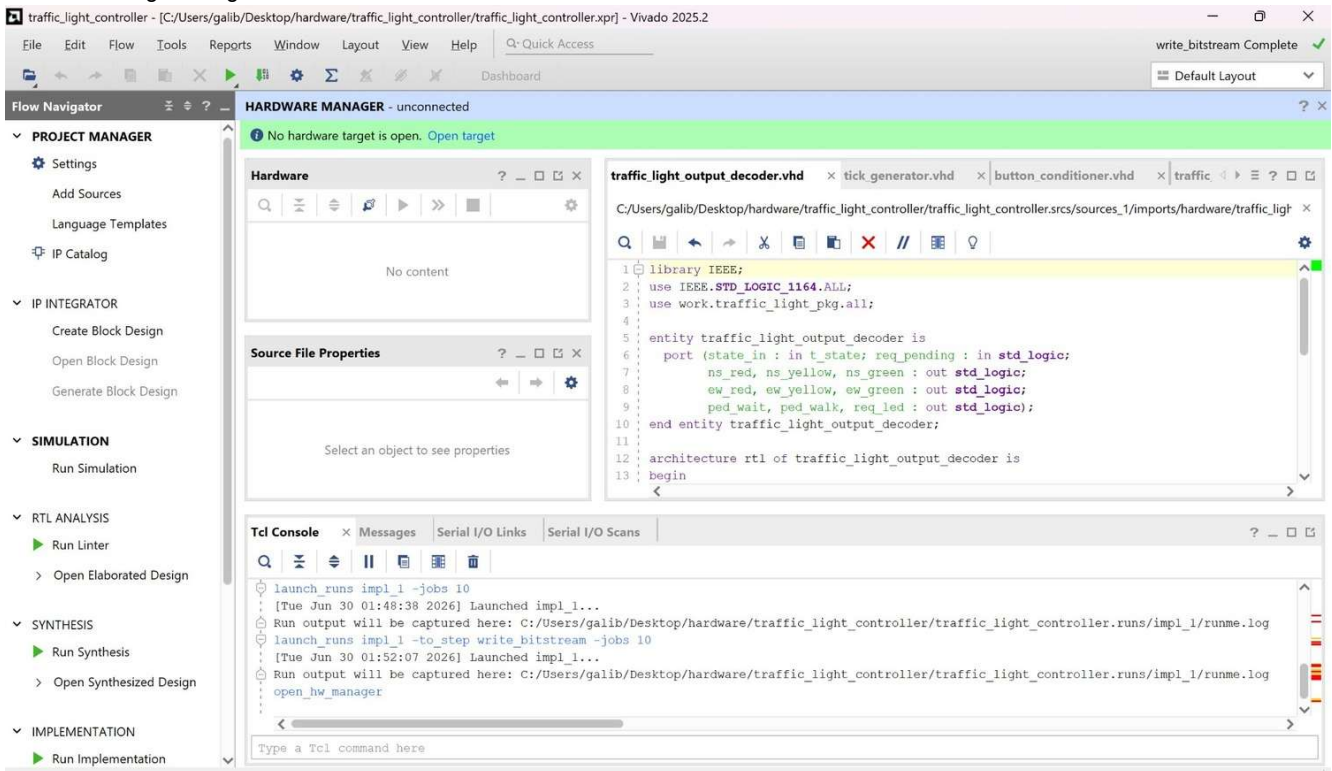


Figure 12: Vivado status showing write_bitstream Complete for the traffic_light_controller project.

8. PCB design and hardware implementation

The supplied PCB design implements the external traffic-light controller interface. It contains LED indicators, push buttons, a 12-pin header, voltage regulation and decoupling capacitors. The PCB files were supplied as Altium documents and an output package containing Gerber files, a PDF view, a BOM spreadsheet and a 3D view.

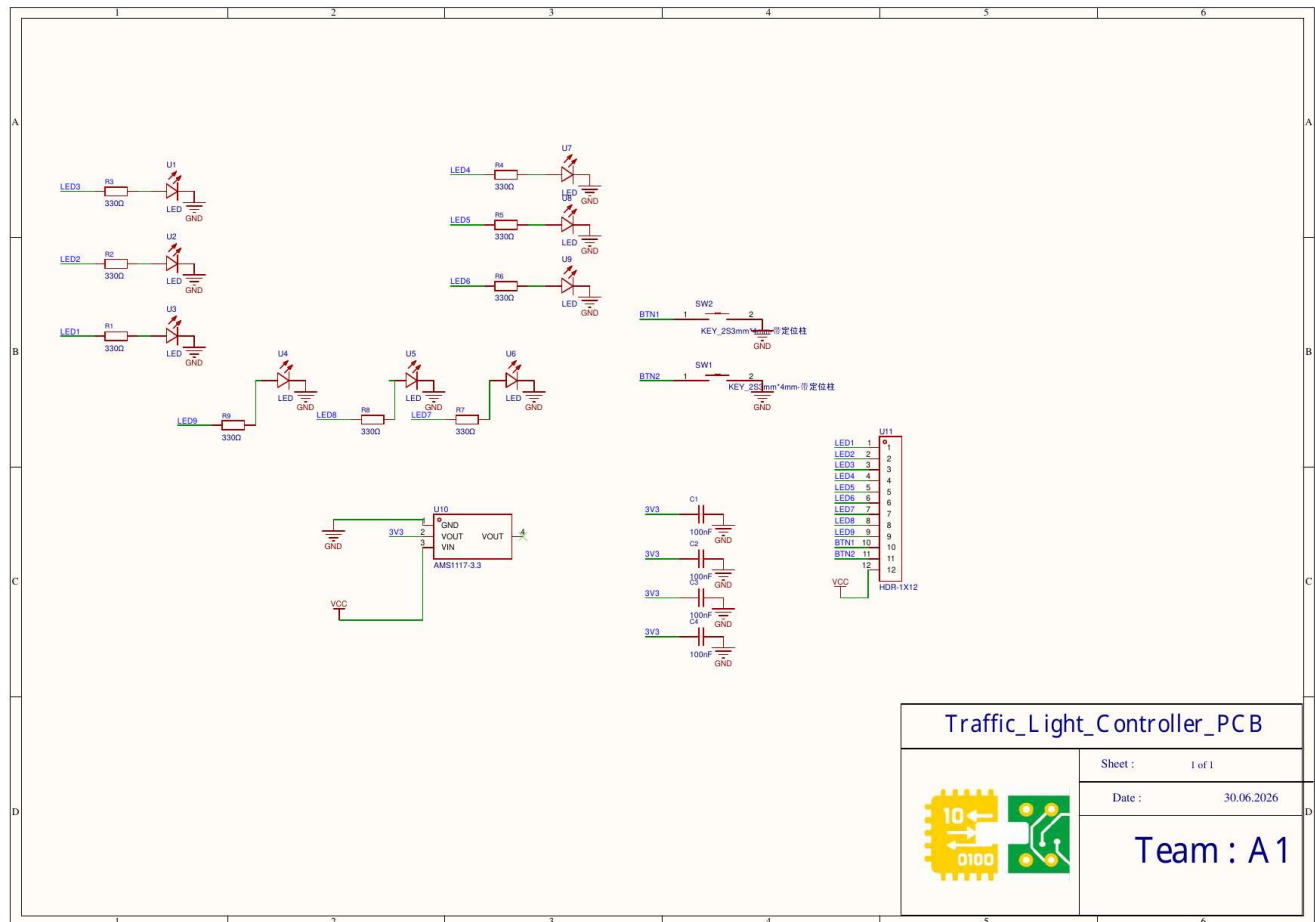


Figure 13: Schematic output showing LED channels, switches, 3.3 V regulator, decoupling capacitors and 1x12 interface header.

8.1 Schematic blocks

Block	Components	Purpose
LED indicators	U1-U9 and R1-R9, 330 ohm resistors	Represent NS red/yellow/green, EW red/yellow/green, pedestrian WAIT/WALK and request status.
Push-button inputs	SW1 and SW2	Provide reset and pedestrian-request inputs to the interface.
Board connector	U11, HDR-1X12	Carries LED, button, power and ground signals between the PCB and the FPGA/prototype interface.
Power regulation	U10, AMS1117-3.3	Provides a local 3.3 V supply rail.
Decoupling	C1-C4, 100 nF capacitors	Stabilizes the local supply and reduces switching noise.

8.2 PCB layout and 3D view

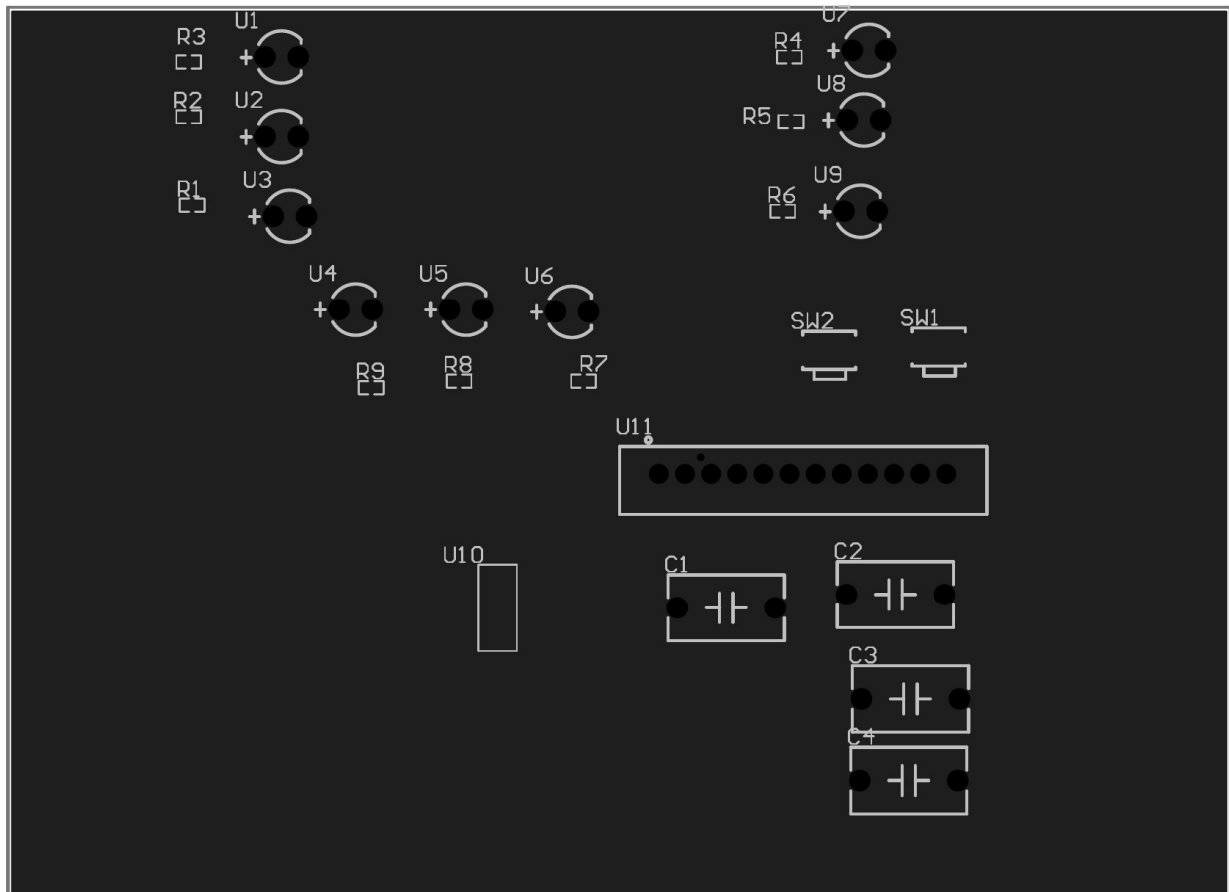


Figure 14: PCB layout/top-view output from the supplied Altium project PDF.

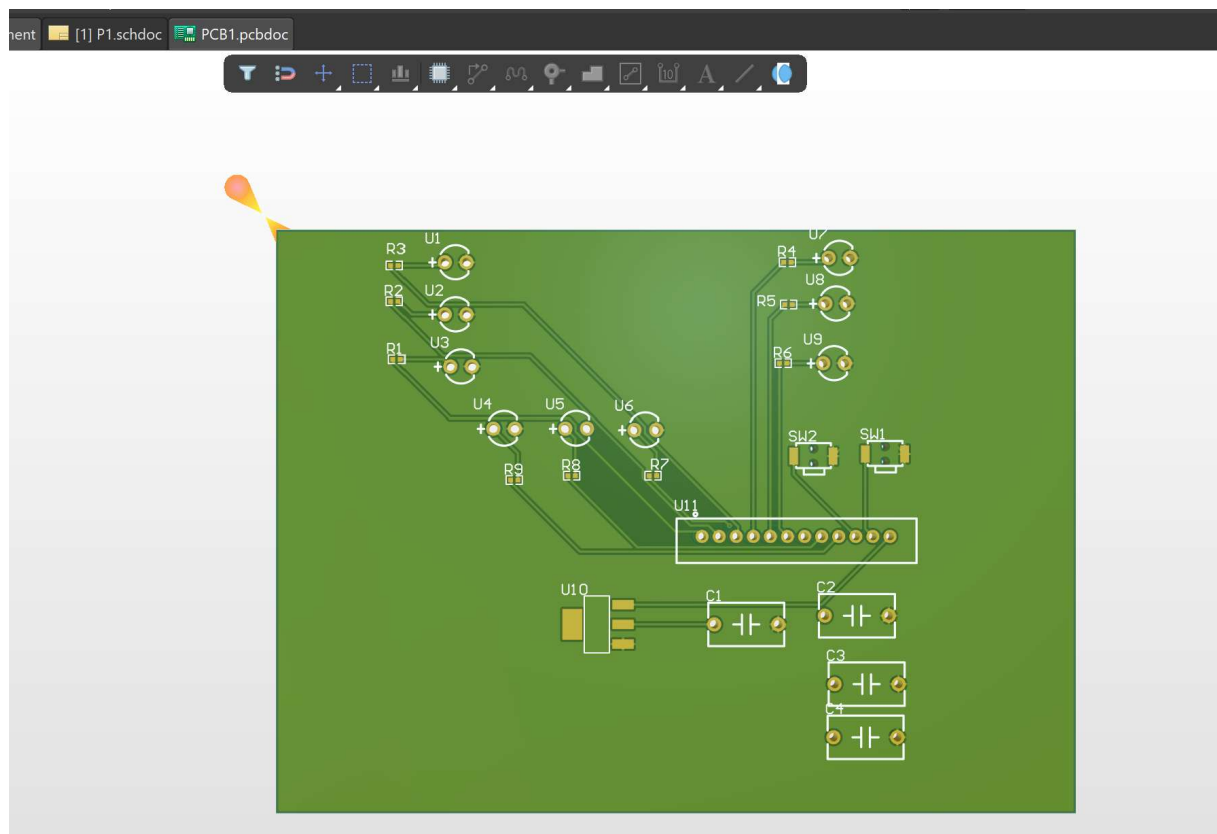


Figure 15: 3D PCB view showing placement of LEDs, buttons, header and power components.

Board Stack Report

Figure 16: PCB output page showing layer-stack / board output information from the supplied PDF.

8.3 BOM summary

Name	Description / role	Designators	Qty
100 nF capacitor	Supply decoupling capacitor, through-hole footprint.	C1, C2, C3, C4	4
330 ohm resistor	LED current-limiting resistor.	R1-R9	9
Tactile switch	User/reset/pedestrian input push buttons.	SW1, SW2	2
LED	Traffic and pedestrian indicator LEDs.	U1-U9	9
AMS1117-3.3	3.3 V linear regulator.	U10	1
HDR-1X12	Interface header.	U11	1

8.4 Generated PCB outputs

Output	Evidence / file	Purpose
Gerber files	Top/bottom copper, solder mask, top/bottom legend and profile Gerbers	Manufacturing data for PCB fabrication.
Status report	Status Report.Txt	Reports that 9 output files were generated at 21:09:13 on 30/06/2026.
PDF output	Traffic_Light_Controller.pdf	Schematic, PCB top view and PCB stack/report pages.

BOM spreadsheet	Traffic_Light_Controller.xlsx	Component descriptions, designators, footprints, library references and quantities.
3D view	PCB3D.png	Visual mechanical/placement evidence.

9. Discussion, problems faced and design decisions

Button conditioning: The pedestrian button is asynchronous and may bounce, so it is not connected directly to the FSM. A synchronizer, debounce counter and edge detector were implemented.

Safe pedestrian insertion: A request does not interrupt an active vehicle green/yellow phase. It is stored and then served at S6_ALL_RED_2, so WALK is only active when both vehicle directions are red.

Timing and simulation speed: Hardware timing uses a 100 MHz clock and one-second tick. For faster simulation, the same architecture can use smaller timing generics/constants.

Timing constraints: The routed design meets the 100 MHz clock requirement, but the timing report warns about missing input or output delays. These should be added if a stricter production-style constraints file is required.

Hardware programming evidence: The uploaded screenshot proves bitstream generation. Because the hardware manager was unconnected, a final report could add a photo/video of the programmed board as additional validation.

PCB integration: The PCB is suitable as an external interface board for the LED and button signals. A final check should confirm that connector pin order matches the FPGA XDC constraints and physical wiring.

10. Conclusion and future improvements

The FPGA-based traffic-light controller with pedestrian request button has been specified, implemented in modular VHDL, simulated with a behavioral testbench and processed through Vivado synthesis, implementation, timing analysis, utilization reporting, power reporting and bitstream generation. The results show that the design uses a very small fraction of the Nexys A7-100T resources and meets the specified 100 MHz timing constraint.

The PCB work provides a project-specific hardware interface with LEDs, buttons, a header, regulator and decoupling capacitors. The generated PCB output package includes schematic/layout evidence, a 3D view, BOM and Gerber manufacturing files. The project therefore demonstrates the complete Hardware Engineering Lab workflow from concept and raw sketch to VHDL implementation, FPGA evidence and PCB documentation.

10.1 Future improvements

- Add a programmed-board photo or short demonstration video to complement the bitstream-generation screenshot.
- Add input/output delay constraints for a more complete timing constraints file.
- Implement blinking pedestrian WAIT during S8_PED_CLEAR.
- Add a seven-segment countdown for the pedestrian WALK interval.
- Add a night mode with flashing yellow vehicle outputs.

- Label the PCB silkscreen more clearly by separating NS, EW and pedestrian LED groups.

Appendix A. VHDL source-file summary

This appendix provides compact excerpts from the uploaded VHDL source files. The full source files remain separate project files and should be stored with the Vivado project.

traffic_light_pkg.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

package traffic_light_pkg is
  type t_state is (
    S0_RESET, S1_NS_GREEN, S2_NS_YELLOW, S3_ALL_RED_1,
    S4_EW_GREEN, S5_EW_YELLOW, S6_ALL_RED_2,
    S7_PED_WALK, S8_PED_CLEAR
  );
  constant C_NS_GREEN_TIME : integer := 5;
  constant C_EW_GREEN_TIME : integer := 5;
  constant C_YELLOW_TIME   : integer := 2;
  constant C_ALLRED_TIME   : integer := 1;
  constant C_WALK_TIME     : integer := 4;
  constant C_PEDCLEAR_TIME : integer := 2;
  constant C_STARTUP_TIME  : integer := 1;
end package traffic_light_pkg;
```

tick_generator.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tick_generator is
  generic (G_CLK_FREQ_HZ : integer := 100_000_000);
  port (clk : in std_logic; rst : in std_logic; tick_1s : out std_logic);
end entity tick_generator;

architecture rtl of tick_generator is
  signal r_counter : integer range 0 to 100_000_000 - 1 := 0;
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        r_counter <= 0; tick_1s <= '0';
      elsif r_counter = G_CLK_FREQ_HZ - 1 then
        r_counter <= 0; tick_1s <= '1';
      else

```

```

    r_counter <= r_counter + 1; tick_1s <= '0';
end if;
end if;
end process;
end architecture rtl;

```

button_conditioner.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity button_conditioner is
    generic (G_DEBOUNCE_TICKS : integer := 20);
    port (clk : in std_logic; rst : in std_logic;
          btn_raw : in std_logic; btn_pulse : out std_logic);
end entity button_conditioner;

architecture rtl of button_conditioner is
    signal r_sync0, r_sync1 : std_logic := '0';
    signal r_debounce_cnt : integer range 0 to 20 := 0;
    signal r_stable : std_logic := '0';
    signal r_prev_stable : std_logic := '0';
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                r_sync0 <= '0'; r_sync1 <= '0';
                r_debounce_cnt <= 0; r_stable <= '0';
                r_prev_stable <= '0'; btn_pulse <= '0';
            else
                r_sync0 <= btn_raw; r_sync1 <= r_sync0;
                if r_sync1 = r_stable then
                    r_debounce_cnt <= 0;
                else
                    if r_debounce_cnt = G_DEBOUNCE_TICKS then
                        r_stable <= r_sync1; r_debounce_cnt <= 0;
                    else
                        r_debounce_cnt <= r_debounce_cnt + 1;
                    end if;
                end if;
                r_prev_stable <= r_stable;
                if r_stable = '1' and r_prev_stable = '0' then
                    btn_pulse <= '1';
                else

```



```

        btn_pulse <= '0';
    end if;
end if;
end if;
end process;
end architecture rtl;

```

traffic_light_fsm.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.traffic_light_pkg.all;

entity traffic_light_fsm is
    port (clk : in std_logic; rst : in std_logic;
          tick_1s : in std_logic; ped_btn_pulse : in std_logic;
          state_out : out t_state; request_pending : out std_logic);
end entity traffic_light_fsm;

architecture rtl of traffic_light_fsm is
    signal r_state : t_state := S0_RESET;
    signal r_timer : integer := 0;
    signal r_req_pending : std_logic := '0';
begin
    state_out <= r_state;
    request_pending <= r_req_pending;

    process(clk)
    begin
        ...
        if tick_1s = '1' then
            r_timer <= r_timer + 1;
            case r_state is
                when S0_RESET =>
                    if r_timer >= C_STARTUP_TIME-1 then r_state <= S1_NS_GREEN; r_timer <= 0; end if;
                when S1_NS_GREEN =>
                    if r_timer >= C_NS_GREEN_TIME-1 then r_state <= S2_NS_YELLOW; r_timer <= 0; end
            if;
                when S2_NS_YELLOW =>
                    if r_timer >= C_YELLOW_TIME-1 then r_state <= S3_ALL_RED_1; r_timer <= 0; end if;
                when S3_ALL_RED_1 =>
                    if r_timer >= C_ALLRED_TIME-1 then r_state <= S4_EW_GREEN; r_timer <= 0; end if;
                when S4_EW_GREEN =>
                    if r_timer >= C_EW_GREEN_TIME-1 then r_state <= S5_EW_YELLOW; r_timer <= 0;
            end if;
        end if;
    end process;
end architecture rtl;

```

```

when S5_EW_YELLOW =>
  if r_timer >= C_YELLOW_TIME-1 then r_state <= S6_ALL_RED_2; r_timer <= 0; end if;
when S6_ALL_RED_2 =>
  if r_timer >= C_ALLRED_TIME-1 then
    r_timer <= 0;
    if r_req_pending = '1' then r_state <= S7_PED_WALK;
    else r_state <= S1_NS_GREEN; end if;
  end if;
when S7_PED_WALK =>
  r_req_pending <= '0';
  if r_timer >= C_WALK_TIME-1 then r_state <= S8_PED_CLEAR; r_timer <= 0; end if;
when S8_PED_CLEAR =>
  if r_timer >= C_PEDCLEAR_TIME-1 then r_state <= S1_NS_GREEN; r_timer <= 0; end if;
when others => r_state <= S0_RESET; r_timer <= 0;
end case;
end if;
end if;

```

traffic_light_output_decoder.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.traffic_light_pkg.all;

entity traffic_light_output_decoder is
  port (state_in : in t_state; req_pending : in std_logic;
        ns_red, ns_yellow, ns_green : out std_logic;
        ew_red, ew_yellow, ew_green : out std_logic;
        ped_wait, ped_walk, req_led : out std_logic);
end entity traffic_light_output_decoder;

architecture rtl of traffic_light_output_decoder is
begin
  req_led <= req_pending;
  process(state_in)
  begin
    ns_red<='1'; ns_yellow<='0'; ns_green<='0';
    ew_red<='1'; ew_yellow<='0'; ew_green<='0';
    ped_wait<='1'; ped_walk<='0';
    case state_in is
      when S1_NS_GREEN => ns_red<='0'; ns_green<='1';
      when S2_NS_YELLOW => ns_red<='0'; ns_yellow<='1';
      when S4_EW_GREEN => ew_red<='0'; ew_green<='1';
      when S5_EW_YELLOW => ew_red<='0'; ew_yellow<='1';
      when S7_PED_WALK => ped_wait<='0'; ped_walk<='1';
    end case;
  end process;
end architecture rtl;

```

```

    when others => null;
  end case;
end process;
end architecture rtl;

```

traffic_light_top.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.traffic_light_pkg.all;

entity traffic_light_top is
  port (clk, rst, btn_ped : in std_logic;
        ns_red, ns_yellow, ns_green : out std_logic;
        ew_red, ew_yellow, ew_green : out std_logic;
        ped_wait, ped_walk, req_led : out std_logic);
end entity traffic_light_top;

architecture structural of traffic_light_top is
  signal s_tick_1s, s_btn_pulse, s_req_pending : std_logic;
  signal s_state : t_state;
begin
  u_tick : entity work.tick_generator
    generic map(G_CLK_FREQ_HZ => 100_000_000)
    port map(clk=>clk, rst=>rst, tick_1s=>s_tick_1s);
  u_btn : entity work.button_conditioner
    generic map(G_DEBOUNCE_TICKS => 20)
    port map(clk=>clk, rst=>rst, btn_raw=>btn_ped, btn_pulse=>s_btn_pulse);
  u_fsm : entity work.traffic_light_fsm
    port map(clk=>clk, rst=>rst, tick_1s=>s_tick_1s,
             ped_btn_pulse=>s_btn_pulse, state_out=>s_state,
             request_pending=>s_req_pending);
  u_dec : entity work.traffic_light_output_decoder
    port map(state_in=>s_state, req_pending=>s_req_pending,
             ns_red=>ns_red, ns_yellow=>ns_yellow, ns_green=>ns_green,
             ew_red=>ew_red, ew_yellow=>ew_yellow, ew_green=>ew_green,
             ped_wait=>ped_wait, ped_walk=>ped_walk, req_led=>req_led);
end architecture structural;

```

traffic_light_fsm_tb.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.traffic_light_pkg.all;

```

```

entity traffic_light_fsm_tb is
end entity traffic_light_fsm_tb;

architecture sim of traffic_light_fsm_tb is
  constant C_CLK_PERIOD : time := 10 ns;
  signal tb_clk, tb_rst, tb_btn_ped : std_logic := '0';
  signal tb_ns_red, tb_ns_yel, tb_ns_grn : std_logic;
  signal tb_ew_red, tb_ew_yel, tb_ew_grn : std_logic;
  signal tb_ped_wait, tb_ped_walk, tb_req_led : std_logic;
begin
  tb_clk <= not tb_clk after C_CLK_PERIOD/2;
  UUT : entity work.traffic_light_top
    port map(clk=>tb_clk, rst=>tb_rst, btn_ped=>tb_btn_ped,
      ns_red=>tb_ns_red, ns_yellow=>tb_ns_yel, ns_green=>tb_ns_grn,
      ew_red=>tb_ew_red, ew_yellow=>tb_ew_yel, ew_green=>tb_ew_grn,
      ped_wait=>tb_ped_wait, ped_walk=>tb_ped_walk, req_led=>tb_req_led);
  stim_proc : process
  begin
  ...
    wait for 150 us;
    report "TB-04 to TB-08: Check waveforms.";
    wait;
  end process;

  process(tb_clk)
  begin
    if rising_edge(tb_clk) then
      assert NOT(tb_ns_grn='1' AND tb_ew_grn='1')
        report "SAFETY FAIL: NS+EW green together!" severity FAILURE;
      assert NOT(tb_ped_walk='1' AND (tb_ns_grn='1' OR tb_ew_grn='1'))
        report "SAFETY FAIL: PED_WALK with vehicle green!" severity FAILURE;
    end if;
  end process;
end architecture sim;

```

Appendix B. Nexys A7 pin mapping and XDC draft

The following draft XDC block is based on the raw top-level diagram and should be checked against the exact Nexys A7 master XDC and final Vivado constraints file before submission.

```

## Clock
create_clock -period 10.000 -name clk -waveform {0.000 5.000} [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

```

Push buttons

```
set_property PACKAGE_PIN M18 [get_ports btn_ped] # BTNL pedestrian request
set_property PACKAGE_PIN N17 [get_ports rst] # BTNC reset
set_property IOSTANDARD LVCMOS33 [get_ports {btn_ped rst}]
```

LEDs

```
set_property PACKAGE_PIN H17 [get_ports ns_red] # LD0
set_property PACKAGE_PIN K15 [get_ports ns_yellow] # LD1
set_property PACKAGE_PIN J13 [get_ports ns_green] # LD2
set_property PACKAGE_PIN N14 [get_ports ew_red] # LD3
set_property PACKAGE_PIN R18 [get_ports ew_yellow] # LD4
set_property PACKAGE_PIN V17 [get_ports ew_green] # LD5
set_property PACKAGE_PIN U17 [get_ports ped_wait] # LD6
set_property PACKAGE_PIN U16 [get_ports ped_walk] # LD7
set_property PACKAGE_PIN V16 [get_ports req_led] # LD8
set_property IOSTANDARD LVCMOS33 [get_ports {ns_red ns_yellow ns_green ew_red ew_yellow
ew_green ped_wait ped_walk req_led}]
```

Appendix C. BOM summary

The BOM information below is summarized from the uploaded Traffic_Light_Controller.xlsx spreadsheet.

Name	Full designators	Footprint	LibRef	Qty
100nF	C1, C2, C3, C4	CAP-TH_L9.0-W5.0-P7.50-D0.6-1	CFC2G104JD1IJ5085100	4
330 ohm	R1, R2, R3, R4, R5, R6, R7, R8, R9	R0402	0402WGF3300TCE	9
KEY_2S3mm*4mm	SW1, SW2	KEY-SMD_L4.0-W3.7-LS4.8	KEY_2S3mm*4mm	2
LED	U1, U2, U3, U4, U5, U6, U7, U8, U9	LED-TH_BD4.0-P2.54-FD_RED	LED	9
AMS1117-3.3	U10	SOT-223-3_L6.5-W3.4-P2.30-LS7.0-BR	AMS1117-3.3_C6186	1
HDR-1X12	U11	CONN-TH_HDR-1X12	HDR-1X12	1

Appendix D. Final review checklist

Checklist item	Status / notes
Fill in matriculation numbers and email addresses for all team members.	[Pending team input]

Confirm that task-division table matches the actual work done.	[Pending team review]
Confirm final timing values used for the FPGA demonstration.	[Partially complete: VHDL constants are documented]
Verify connector pin order between PCB header and FPGA/XDC pin mapping.	[Pending hardware review]
Add final board-programming photo/video if available.	[Pending physical demo evidence]
Check PCB title block/team label and update if needed.	[Pending Altium review]
Confirm final PDF page layout before submission.	[Completed by render QA for this draft]